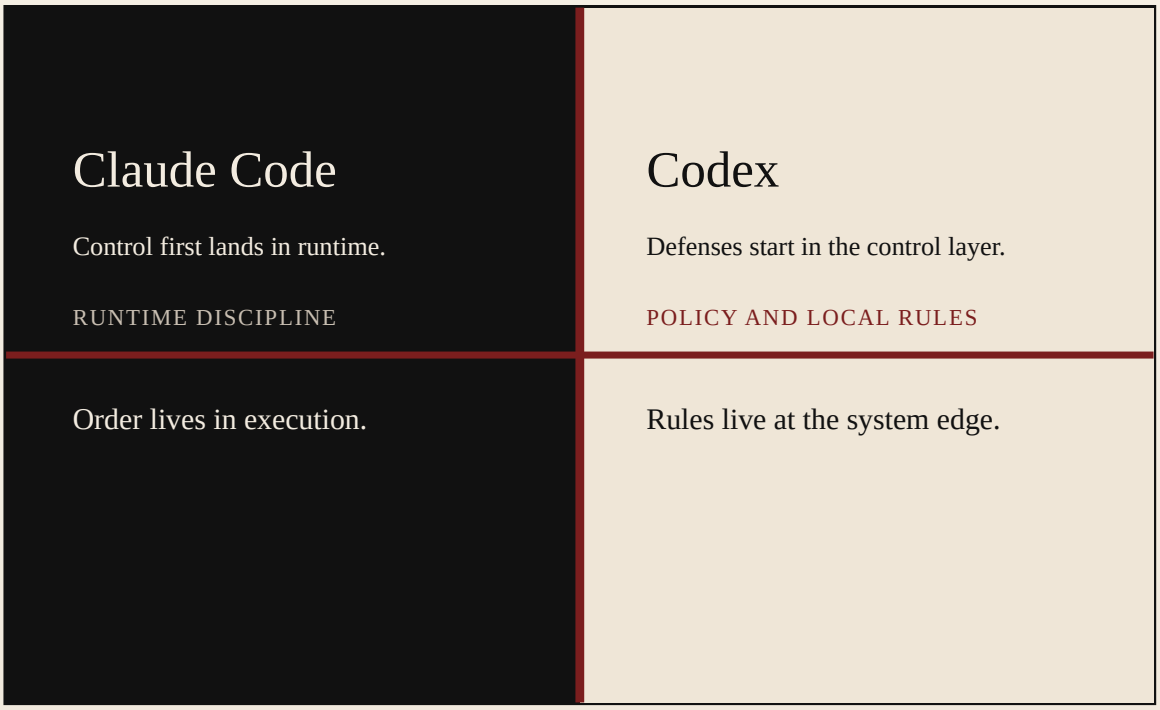

The Harness Design Philosophies of Claude Code and Codex

Convergence through different roads, or separate branches?

System builders are just people
who write inevitable damage
into control flow early enough
that it cannot govern them later.

CONTROL / LOOP / POLICY / STATE / LOCAL GOVERNANCE / VERIFICATION

A comparative engineering notebook on control layers, state governance, tool boundaries, and organizational discipline

agentway.dev

From using agents to shipping an agent PoC

https://agentway.dev

2026-04-01 · rev fbf2b4

Contents

Introduction

Subtitle: convergence, divergence, and the engineering choices behind two coding-agent systems.

This is the English edition of the comparative book. It is structured to mirror the Chinese edition while emphasizing the points that matter to readers making architectural choices in English-language teams.

What this collection offers

- A reading map that explains how this book sits beside the first volume.
- A series of chapters that trace the same harness concerns (control plane, continuity, tool policy, hooks, delegation, and verification) through both Claude Code and Codex.
- Appendices that map sources and boil the comparison down to practical checklists.

Use the navigation on the left to follow the chapter order above. If you just want the conclusion, start at Chapter 7.

Reading Map: How to Read Book One and This Comparative Book Together

If you treat both directories as "books about AI coding agents," the easiest misread is to assume they are duplicate documents. They are not. Their division of labor is clear.

Book One is single-system dissection. It uses Claude Code as a specimen to explain why a controllable agent must grow organs such as a control plane, query loop, tool permissions, context governance, recovery paths, multi-agent verification, and team institutions. Its central question is: what internal skeleton does a harness need in order to work continuously in real engineering environments?

This comparison book is comparative dissection. It no longer asks only why Claude Code is designed this way, but puts Claude Code and Codex side by side to compare how each admits model unreliability and places order at different layers. Its central question is: when both systems are building a harness, which choices are consensus and which are just different engineering routes?

There is now one more use for that comparison: once you carry these judgments into third-party harnesses, it becomes easier to spot a common failure mode. Many systems also advertise memory, skills, compaction, and multi-agent support, yet their context-governance axis still amounts to pushing more text into prompt first and truncating or rescuing later. That route looks like "more information," but in practice it often burns more tokens while weakening working semantics.

In other words, you can read them as two sequential steps in one research program:

- Step 1: extract general principles of Harness Engineering in Book One.
- Step 2: observe how those principles land differently across two concrete systems in this comparison book.

Start with Book One: Nine Structural Judgments from Claude Code

Book One compresses to nine: a harness first stops the model from damaging engineering environments; prompt is part of the control plane; the query loop is the agent's heartbeat; tools are execution interfaces constrained by approval, orchestration, and interrupt semantics; more context is not always better, memory / `CLAUDE.md` / compact are budget-governance mechanisms; errors belong on the main path, recovery is first-class; multi-agent value is role partitioning plus independent verification; team rollout must crystallize rules into reusable institutions; together these become a stable Harness Engineering principle checklist.

Reading only Book One leaves one strong impression: Claude Code's character is runtime governance —it cares first about how the session keeps running, how tools avoid trouble, how recovery avoids dead loops, and how verification avoids ritualism.

Then Read This Comparison Book: Same Problem, Different Starting Points

Based on that foundation, this book splits comparison into explicit layers.

Control Plane

Claude Code behaves more like dynamic prompt assembly. A lot of order is packed into runtime prompt composition, session state, and context governance.

Codex behaves more like explicit control-layer engineering. It modularizes and types instruction fragments, approval policies, tool schema, thread, rollout,

and hooks as much as possible.

Continuity

Claude Code places continuity in the main loop and emphasizes query-loop heartbeat discipline.

Codex distributes continuity across thread, rollout, and state bridge, emphasizing structured state ownership and recovery.

Tools and Permissions

Claude Code leans toward runtime constraints: approval at call time, interrupts, and preventing risky actions from landing directly.

Codex leans toward policy language and tool contracts: schema, approval policy, sandbox, and exec policy as explicit organs.

Local Governance

Claude Code tends to absorb local experience into field memory: `CLAUDE.md`, memory, skills, and workflow constraints.

Codex tends to mount local institutions onto structured injection and event systems: instructions, skills, hooks, and explicit tool boundaries.

Multi-Agent and Verification

Claude Code emphasizes multi-agent role partitioning at runtime, and insists verification must be independent from implementation.

Codex emphasizes explicit delegation, persistent state, and tool-mediated collaboration so verification becomes a trackable capability, rather than remaining a polite final gesture.

What Becomes Clear When You Read Them Together

Putting Book One and this comparative volume together yields three fuller conclusions.

First, the comparison is centered mainly on the harness

Both books point to the same fact: the core challenge of AI coding systems is to keep models from losing control in terminals, filesystems, permission boundaries, and team institutions. Gains in eloquence matter, but they sit downstream of that problem.

Second, the main difference between Claude Code and Codex is where order is placed

Claude Code is closer to a system grown from runtime incident pressure, prioritizing continuity, recovery, and field governance.

Codex is closer to a system grown from explicit structural design, prioritizing control-layer naming, policy expression, boundary clarity, and composability.

Third, followers usually gain more by identifying their own dominant uncertainty

If your biggest pain is long-session instability, brittle recovery, and skipped verification, start with the runtime discipline emphasized in Book One.

If your biggest pain is scattered rules, fuzzy permission boundaries, unstable tool contracts, and non-reproducible team behavior, start with the explicit-control-layer lessons this comparison extracts from Codex.

And if you encounter a system whose continuity depends mainly on stacking bootstrap text, identity files, skill catalogs, and workspace prose into prompt, do not be too impressed by the apparent fullness of context. In many cases, that points to an intermediate state that has not yet truly governed context, rather than a mature third road.

Recommended Reading Order

If you are new to this material set, use this order:

1. Read the Preface first to confirm that this material is comparing where order is located, not merely listing features.
2. Read Chapter 1 Why Put Claude Code and Codex Side by Side to establish problem framing.
3. Then read Chapters 2 through 6 in sequence across five axes: control plane, continuity, tool governance, local institutions, and multi-agent verification.
4. If you only want a quick synthesis, jump to Chapter 7 Converging Destinations, Diverging Branches.
5. If your goal is to build systems yourself, finish with Chapter 8 If You Build One Yourself: Who to Learn from, and What to Learn First. That chapter now also includes a three-path diagram showing why some harnesses still feel expensive and disorderly even when their context looks full.

One-Sentence Summary

Book One explains why a controllable agent must grow this way.

This comparative book explains why two serious harness systems still grow differently.

Preface: Two harnesses, not one accessory pack

Comparing two AI coding harnesses is easy to get wrong if you only line up feature checkboxes. Both Claude Code and Codex have skills, sandboxes, and sub-agents. But seeing shared terminology does not mean their skeletons are the same. It is like noting both cities have bridges—the real question is which river they are trying to cross.

This book aims to compare the bones, not the labels.

Both systems admit something that marketing often sweeps under the rug: a model cannot be trusted to execute without constraints. Shells, filesystems, permissions, tool calls, teams, long sessions, and recovery paths are messy realities. A harness is that messy reality; it is the apparatus that keeps an unreliable model from burning the environment down. When that harness exists, you no longer measure success by how eloquently the model speaks—you measure it by where and how the system places guardrails.

In the first volume I treated Claude Code as a specimen and extracted the general principles of Harness Engineering. That was single-system anatomy. Here the goal is to place Claude Code beside another serious harness with a different lineage. Only in comparison do many design choices reveal themselves as one engineering path among many.

Codex is worth comparing precisely because it is not a clone. A glance at its `core/src/lib.rs` shows a deliberate program: threads, rollouts, state bridges, instructions, skills, hooks, sandboxing, exec policy, tools—each one composed into an explicit module. The ambition is to make the control layer composable, serializable, and policy-aware instead of hiding it inside a bowl of runtime intuition.

Claude Code, by contrast, feels like something grown under the pressure of

its runtime loop. Look at `src/query.ts` and the surrounding compacting, tool orchestration, permission handling, interrupts, and recovery logic. Most of its magic answers the question "how does this round hand off safely to the next round?" The harness first has to keep running continuously; once continuity is stable, then one can polish the rules between phases.

Both systems agree that models are unreliable. That shared conviction matters. Once you admit the model cannot be left inhabiting shell, files, permissions, and long conversations on its own, a harness must grow—prompt stacking, state persistence, approval, context governance, recovery flows, verification, and local conventions. Only the placement of those organs differs.

This book does not reproduce Claude Code or Codex source, and it does not transcribe long implementation excerpts. That boundary is not mysterious: respect for proprietary code, as well as a desire to keep the argument in the engineering analysis. What follows is a comparison of how these systems think about themselves, not a copy-paste of their implementations.

If I had to summarize in one sentence:

> Claude Code and Codex are aligned not because they both call tools, but because neither is willing to treat the model as a free-moving executor.

As for their differences, they are more interesting.

Claude Code approaches harnessing through runtime discipline. It worries about session rupture, tool chaining, compact behavior, user interjections, and what happens if a forked agent dies mid-conversation. It has the feel of someone who has cleaned up after messy shifts in a real operations center—preferring to solve "smart" problems through craftsmanship in control and recovery flows.

Codex approaches it through structured control. Instruction fragments, threads, approval policies, tool schemas, hook events, and exec policies are all spelled out. It values naming, explicit boundaries, and configuration. Codex does not rely on tacit understanding; it prefers to declare marker types and inject them deterministically.

Both routes are valid. They are just valid in different ways.

This is not an article about winners and losers. Valuable engineering comparison helps identify path dependencies, not rank execution speed. The way

you choose to constrain an unreliable model determines what your harness becomes. Where your team places control is where the system will grow its flesh. Instead of asking "what is the best practice?," ask "what uncertainty am I countering, and where am I willing to anchor order?"

The next seven chapters start from that question.

@wquguru
2026.04.01
Claude Code fools-day source leak

P.S. Read the online version at harness-books.agentway.dev/book2-comparing for a richer experience.

Chapter 1: Why we compare Claude Code and Codex

1.1 Because both distrust the model

Calling Claude Code and Codex "models that write code" misses the point. The interesting comparison is not "who has more buttons," it is how each harness admits a fact marketing often disguises: the model cannot be trusted to operate unbounded shell, files, network, or state. It hallucinates, forgets context, and imagines confidence beyond correctness. Once that model touches a terminal, a filesystem, or a multi-round session, the issue is no longer "was the answer right?" but "who cleans up the mess?"

Claude Code looks like a system born from incident reviews. Read `query.ts`, `toolOrchestration.ts`, `compact.ts`, and the surrounding prompts—you see a system that imagines failure, fatigue, and rollback. Codex is equally honest, but it expresses distrust through explicit modules: threads, rollouts, fragment instructions, exec policies, sandboxing, and tool schemas. Each declares its responsibilities instead of leaving them to the model's intuition.

So this comparison is about how each harness domesticates unreliability—not about who can hit more commands.

1.2 Claude Code: runtime-first harnessing

Claude Code's strengths cluster around the main loop because it starts from one question: "The model is already cycling. How do we make sure one round doesn't sabotage the next?"The system is less interested in "pretty prompts"and more interested in prompt layering, compacting, permission gating, interrupts,

tool orchestration, and forked agent life cycles. The work happens at runtime: managing message inflation, tool output rerouting, context trimming, child-agent isolation, and independent verification phases. This harness grows out of real shifts—so it solves "dirty work" through control flow, not just through elegant interfaces.

1.3 Codex: structured control from the start

Codex takes a different path. Its design treats instructions as typed, bounded fragments. `AGENTS.md`, `skills`, and user fragments appear in the prompts as marked sections with start and end tokens. Tools are schema-first objects. Exec policy is a distinct crate with policies, rules, evaluations, and parsers. The emphasis is on making order explicit, composable, and explainable.

This yields two immediate outcomes: the control plane is easier to explain (you can trace why a fragment is present), and it is easier to programmatically evolve (add new visibility, merge, or precedence rules without rewriting the runtime).

1.4 Same question, different starting points

Both systems try to keep an unreliable model from doing unacceptable things. Claude Code begins by asking how a running loop can stay coherent; Codex begins by asking how control information can be turned into explicit, composable structures. Claude Code gravitates toward runtime discipline; Codex gravitates toward typed infrastructure.

1.5 Why the difference matters

The direction a team takes often determines how its culture evolves. Emphasize runtime continuity, and you will obsess over recovery, interruption, state pollution, tool choreography, and long-session reliability. Emphasize explicit control, and you will obsess over instruction boundaries, config hierarchies, tool schemas, policy languages, and persistent thread state.

Both paths are valid. The mistake is mixing them without clarity and losing both runtime discipline and institutional order.

1.6 The take-away

Claude Code and Codex are harness design philosophies, not feature checklists. They converge on the acceptance that models are untrustworthy; they diverge in what they build around that admission. This book unpacks that divergence.

Chapter 2: Two control planes—dynamic prompt layers vs structured fragments

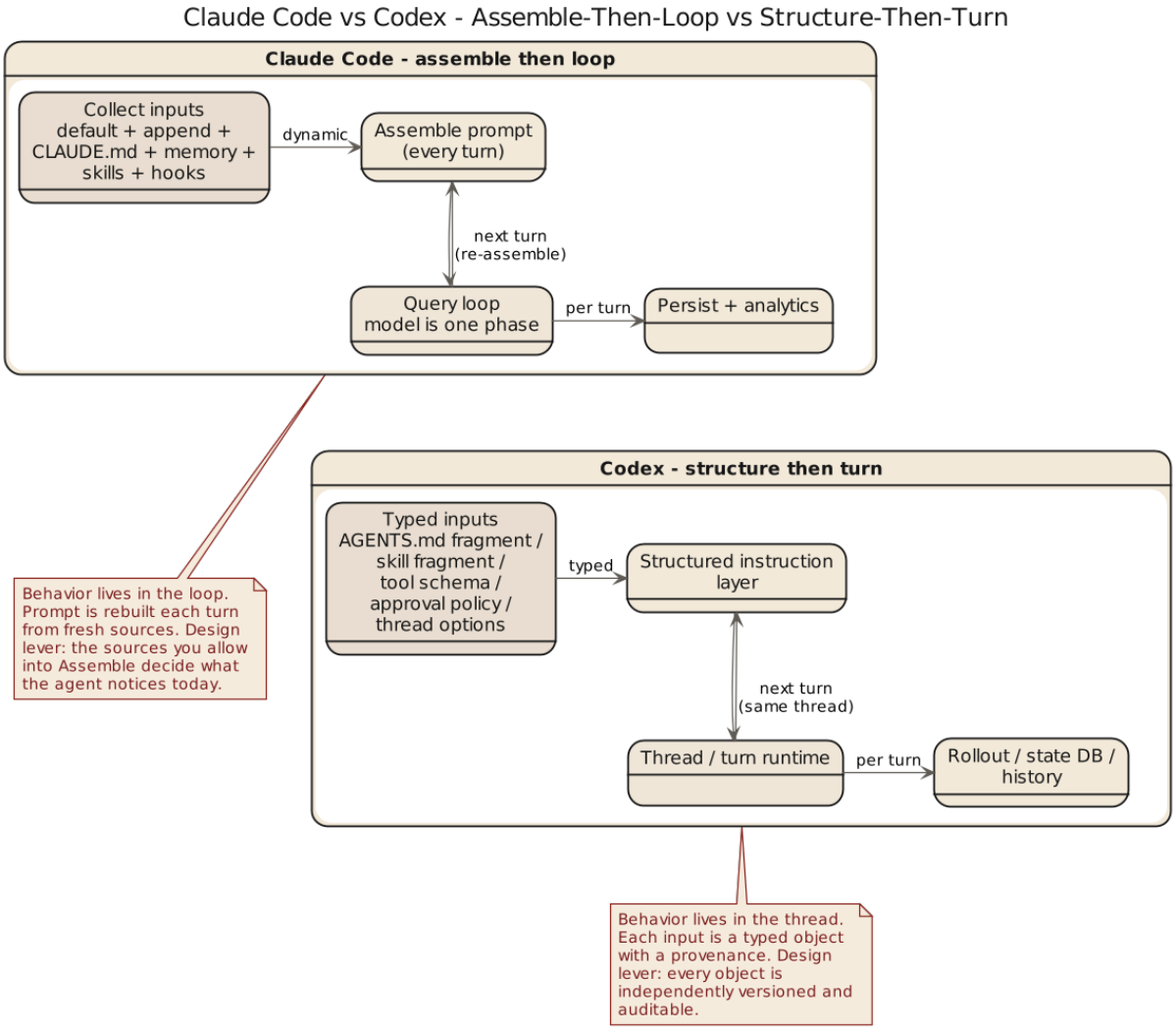

Figure 1: Control plane comparison diagram

2.1 Control is not about tone

Treating prompts as a tone exercise is misleading. Claude Code and Codex both treat prompts as part of a behavioral control plane; the difference is in the mechanism that assembles them. Claude Code assembles dynamically —baseline text, append prompts, agent roles, `CLAUDE.md`, memory, and output styles layered at runtime according to task, tools, and team context; the art is priority, conflict resolution, and how each layer folds into the next. Codex treats instructions as structured fragments —`AGENTS.md`, user messages, and skills become chunks with clear markers, start/end boundaries, and serialization rules, turning instructions from free-form narrative into identifiable contextual units.

2.2 Claude Code's busy assembly line

System prompts here are not fixed documents but a production line: defaults form the foundation, append prompts drop requirements, agent prompts add roles, `CLAUDE.md` and memory inject local conditions. Flexibility lets one loop handle many scenarios, but ordering is critical —wrong ordering dilutes instructions or lets conflicts slip through. Runtime governance is therefore essential: control is constantly injected, overwritten, compressed, or pruned as tasks shift, and the loop recalculates "what matters now" each round. The guiding intuition: control follows the scene —it cannot freeze into static rules.

2.3 Codex's filing-room approach

Codex insists on identifiable fragments. Names like `ContextualUserFragmentDefinition` highlight type, boundaries, wrapping rules, and message transformation. AGENTS, skills, and user instructions are tagged contextual units the system can recognize and manipulate —stronger debuggability, and a path to more programmatic governance because every instruction already fits a type hierarchy.

And this is not merely elegant naming. `fragment.rs` defines constants like `AGENTS_MD_START_MARKER`, `AGENTS_MD_END_MARKER`, `SKILL_OPEN_TAG`, and `SKILL_CLOSE_TAG`; `ContextualUserFragmentDefinition::wrap()` and

`into_message()` turn those fragments into `ResponseItem::Message`. In `user_instructions.rs`, `UserInstructions` serializes the directory into `# AGENTS.md instructions for ...`, while `SkillInstructions` carries explicit `<name>` and `<path>` fields. Codex tries hard not to make the model guess where a rule came from.

Skeleton: two control-plane assemblies

```
// skeleton: Claude Code dynamic assembly (src: constants/prompts.ts, utils/s
system_prompt = concat(
    default_prompt,           // baseline
    append_prompt,            // overlay requirements
    agent_prompt,             // role
    claudemd_layers,          // team / personal / project
    memory_sections,          // session memory
    output_style              // expression discipline
)
// recomputed every loop: memory prefetch, collapse, microcompact, autocompact

// skeleton: Codex fragment assembly (src: instructions/src/fragment.rs, user
for frag in [agents_md, skill, user_instructions]:
    body = ContextualUserFragmentDefinition::wrap(
        START_MARKER, content, END_MARKER,
        meta { source_dir, name, path }
    )
   msg  = frag.into_message()           // -> ResponseItem::Message
    thread.append(msg)
```

Invariants

```
assert every fragment has matching (START_MARKER, END_MARKER)  # markers paire
assert fragment.source ∈ {AGENTS_MD, SKILL, USER}          # type is identifiab
assert precedence(project) > precedence(team) > precedence(default)  # monotor
assert claudemd_layers overlay order = team → personal → project  # later overri
assert child_agents_md enabled ⇒ append scope/precedence notes  # scope is exp
```

2.4 CLAUDE.md vs AGENTS.md

`CLAUDE.md` is a local bulletin board: close to the task directory, paired with memory and skills, good for registering common sense, taboos, and local rules. `AGENTS.md` is pulled into Codex's hierarchy discussion —`docs/agents_md.md` says that even when no `AGENTS.md` is present, enabling `child_agents_md` appends scope and precedence notes. Codex cares not just whether rules exist, but whether their applicability and inheritance are explicitly stated. Claude Code brings local rules into the conversation; Codex brings them into the institution.

2.5 The trade-offs

Runtime assembly is flexible but hard to formalize, leaning on the main loop and engineering judgment; once rules multiply, overlap and semantic dilution become real risks. Structured fragments are explicit but heavier: markers, types, serialization, and injection all need definitions, plus calls on what deserves first-class status. The former grows experience-driven control, the latter grows institutional control —one agile but under-declared, the other clear but carrying ongoing structural cost.

2.6 This chapter's conclusion

> Claude Code views prompts as dynamic runtime builds; Codex views instructions as identifiable fragments.

One feels like a production floor, the other like a bureaucracy. The right choice depends on whether your primary worry is volatile sessions or unclear rule sources. The next chapter goes deeper: does continuity live in the query loop, or in thread, rollout, and state infrastructure?

Chapter 3 Where the Heartbeat Lives: Query Loop Versus Thread, Rollout, and State

3.1 The core of an agent system is continuity

Treating an agent system as "multi-turn chat" is like treating a database as "a patient notebook" —it hides the real architectural problem. Continuity is the hard thing: how this turn picks up from the last, how tool results get merged, how interruption gets closed out, how overgrown context is reorganized, whether failure triggers retry, compaction, or faithful reporting. These answers decide whether a system is really an agent rather than a question-answering interface that happens to support tool calls. Claude Code and Codex diverge here more tellingly than any feature table.

3.2 Claude Code: continuity compressed into the main loop

The axis sits around `query()` and `queryLoop()`, which push many crucial issues into loop state: current message sequence, tool-use context, compact tracking, output-token recovery counters, pending summaries, turn counts, transitions. The answer to "how does an agent stay alive" is runtime-flavored —continuity is maintained mainly by the loop, and the skeleton feels like a self-correcting conversation engine rather than a system driven first by an external state model. The strength is concrete: tool-return ordering, output truncation, prompt-too-long events, history snips, microcompact, and user interjections all happen

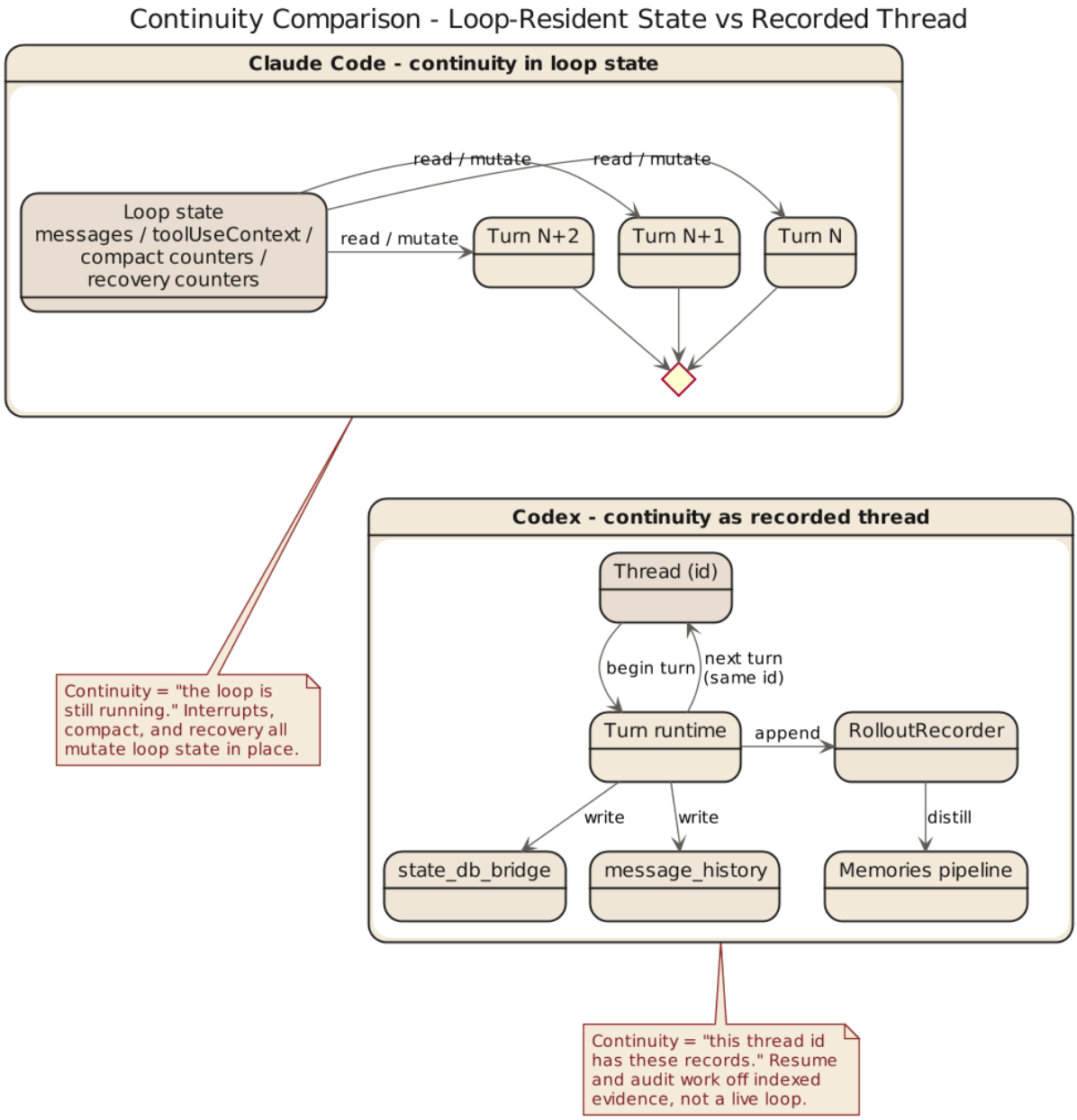

Figure 2: Continuity Comparison

inside the loop, and Claude Code treats them as legitimate loop states rather than avoiding them. The design has rough engineering texture —not always elegant, but often more robust.

3.3 Codex: continuity split across thread, rollout, and state bridges

Codex looks more ledger-like. From `core/src/lib.rs` onward continuity is distributed across `codex_thread`, `thread_manager`, `rollout`, `state_db_bridge`, `state`, and `message_history`.

In `sdk/typescript/src/thread.ts`, `Thread` is already a first-class concept for outside developers: it owns `id`, runs via `runStreamed()` or `run()`, and `thread.started` reports the thread ID back. Turn-level execution conditions —approval policy, working directory, sandbox mode, network access, additional directories —are all explicit parameters tightly coupled to thread execution. Thread sovereignty is literal: `runStreamedInternal()` calls `normalizeInput()` to separate text from images, `createOutputSchemaFile()` to prepare a schema file, then passes `threadId`, `approvalPolicy`, `sandboxMode`, `workingDirectory`, `networkAccessEnabled`, and `additionalDirectories` into `_exec.run()`. When the stream emits `thread.started`, the object updates its `_id`. Thread is not outer packaging —it is the layer turn semantics actually pass through. Continuity is no longer "the loop is still going" but "a thread is being recorded and constrained by an explicit state structure." Rollout, especially, signals that Codex cares about replayability, indexing, persistence, and out-of-session visibility —the system feels more like an execution recorder than a live conversation manager.

3.4 The difference is where state sovereignty lives

Claude Code has state, Codex has loops —the difference is not presence or absence but sovereignty. Claude Code gives more sovereignty to the query loop: "how the session continues" is a runtime-core problem, and many issues are solved directly inside the loop. Codex gives sovereignty to thread and rollout structures: continuity should not remain a by-product of internal control flow;

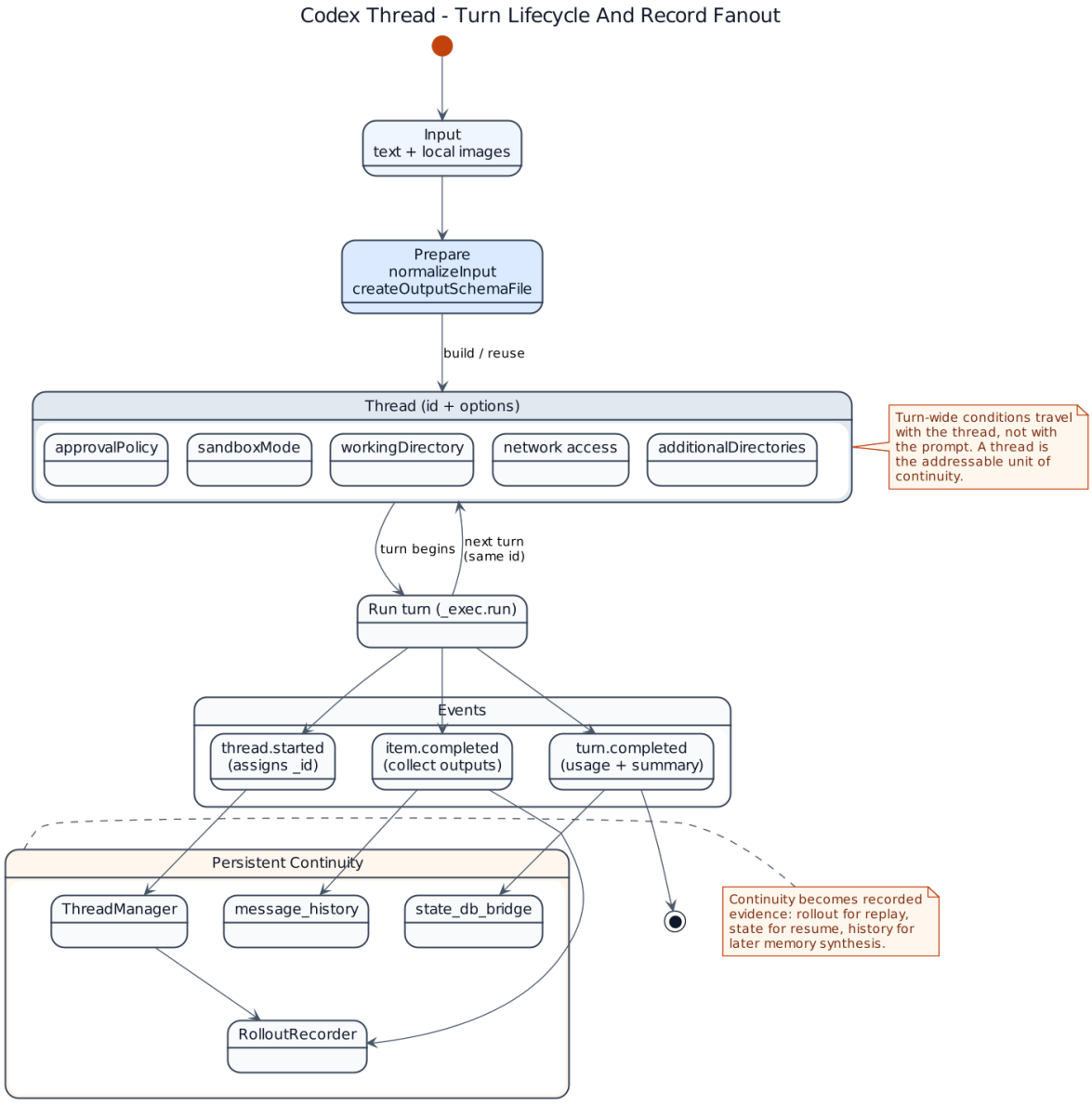

Figure 3: Codex thread-turn-state detail

it should become an explicit fact carried by thread and state infrastructure. That is why `Thread` in the TypeScript SDK already feels like a product concept, and developers think about agent turns through it directly. Claude Code's query loop is more like the engine room —important, but not every user has to organize their mental model around it.

Invariants: continuity sovereignty

```
// Claude Code (src: src/query.ts)
assert loop owns {messages, toolUseContext, compactTracking, turnCount}
assert each loop iteration recomputes "what matters now"
assert pending tool_use closed or synthetic-filled before iteration ends

// Codex (src: core/src/lib.rs, sdk/typescript/src/thread.ts)
assert thread.id stable across runs; rollout records every turn
assert turn-level {approvalPolicy, sandboxMode, workingDirectory} explicit in
assert thread.started event emitted before any tool call
assert state_db_bridge persists before the main loop exits
```

Failure matrix: interrupt with pending tool_use

Event order	Pre-state	Trigger	Claude Code next	Codex next
user interrupt + tool in flight	tool_use unclosed	user abort	synthesize tool_result in loop, close ledger	thread-level abort, write rollout "interrupted" event
model output truncated	max_output_tokens cap hit		raise cap or append meta user msg to continue	thread records truncation, caller restarts turn

Event order	Pre-state	Trigger	Claude Code next	Codex next
prompt too long	session bloated	`prompt_too_long`	collapse / reactive compact / surface inside loop	thread_manager trims history, next turn
process restart	crash mid-session	crash / kill	rely on external PR / Git to rebuild; loop state fragile	rollout + state_db_bridge replay by thread.id
recovery exhausted	consecutive compact failures	breaker threshold	skip stop hook, faithful error	return state-bridge error, keep thread record

3.5 Different implications for recovery and auditability

State placement directly affects recovery and auditability. Claude Code's recovery strength comes from proximity to the scene —many problems are discovered and handled inside the loop itself (reactive compaction, output-token recovery, tool-interruption cleanup), without lifting them into a higher-order state model first. Codex's recovery strength lies in traceability: threads have IDs, rollouts have records, state bridges plus message history provide clearer external structure, making "what exactly happened last turn" answerable from archives rather than runtime recollection. `core/src/lib.rs` read together with the SDK layer makes the archive-minded design obvious — exporting not only `CodexThread` but also `ThreadManager`, `RolloutRecorder`, `state_db_bridge`, and `message_history` near the root module effectively declares continuity as infrastructure, not a side effect of looping. Plain language: Claude Code is closer to an emergency crew on site, Codex closer to a dispatch center with archives —the former is better at keeping execution going, the

latter at explaining how continuity is maintained.

3.6 Different effects on product and team interfaces

Sovereignty in the loop steers teams toward runtime questions: which failures need recovery, which actions should be interruptible, when compaction triggers, how tool results return to the main conversation. Sovereignty in threads and state structures steers teams toward interface and governance questions: thread lifecycle, which events rollout should retain, where the state store lives, how approval policy becomes a turn-level option. Claude Code is closer to making the agent operational first and embedding institutions afterward; Codex is closer to defining institutional interfaces first and letting the agent operate inside them.

3.7 Chapter conclusion

The conclusion can be stated plainly:

> Claude Code places more of continuity inside the query loop, while Codex places more of continuity inside thread, rollout, and state infrastructure.

The former emphasizes runtime heartbeat.

The latter emphasizes a persisted session substrate.

This is not an aesthetic difference. It is a distribution of system power. Whoever owns continuity defines the center of the harness.

The next chapter moves into the hardest layer of all: tools, sandboxes, approvals, and execution policy. Once shell enters the picture, romantic narratives usually leave on their own.

Chapter 4 Tools, Sandboxes, and Policy Languages: Who Stops the Model from Acting Too Fast

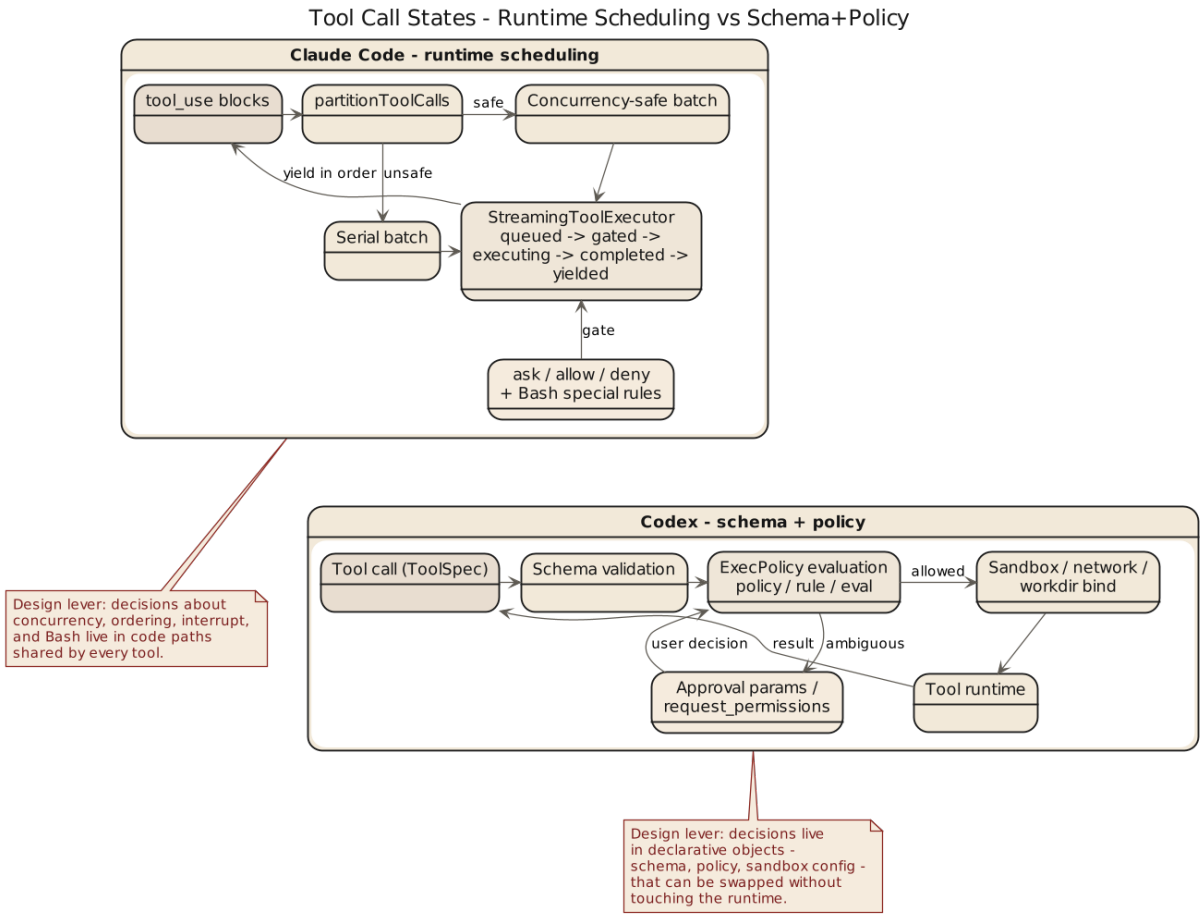

Figure 4: Tool Governance Comparison

4.1 The truly dangerous moment is the start of execution

A model saying the wrong thing wastes time; running the wrong command takes the directory, repository, processes, and workflow down with it. What distinguishes AI coding systems is who owns the final interpretive authority before a tool acts. Both Claude Code and Codex take this seriously in different ways. Claude Code pulls tools into runtime scheduling discipline —`toolOrchestration.ts`, `toolExecution.ts`, `StreamingToolExecutor.ts`, `useCanUseTool.tsx`, Bash-specific prompts, explicit allow/deny/ask semantics —deciding whether a call may run, how, whether concurrently, whether the user rejected, how to interrupt, and how results return to context. Codex turns tools into typed interfaces first —`tools/src/lib.rs` exports tool constructors, `local_tool.rs` defines schemas for `exec_command`, `shell`, `shell_command`, `request_permissions`. Tools in Codex are first a normalized API, only secondarily an execution unit.

4.2 Claude Code: runtime orchestration and dangerous-action constraints

The tool system has a strong field-dispatch feel: concurrency depends on schema and `isConcurrencySafe()`, context modifications preserve replay order, streaming execution must handle interrupts, synthetic results, and UI feedback. Tool invocation is treated as a process with consequences, not a single point action —the harness resembles a site supervisor attached to the model, watching which tool goes first, which can be parallelized, which must serialize, how results are accounted for, and what happens when work is halted halfway. Bash is treated with near-obsessive explicitness —mature systems are usually fussy around the most dangerous interface.

4.3 Codex: tool schemas, approval parameters, and a policy engine

Codex expresses control over risky actions as formal interface constraints. In `local_tool.rs`, `exec_command` explicitly owns fields —`cmd`, `workdir`, `shell`, `tty`, `yield_time_ms`, `max_output_tokens`, `login`, approval-related parameters —rather than accepting a single string command. `shell` / `shell_command` descriptions require `workdir` and warn against careless `cd`. Correct usage is encoded in the tool definition itself. Approvals and escalation are explicit parameters, `request_permissions` is a separate tool, and `execpolicy` is its own crate —`Policy`, `Rule`, `Evaluation`, `Decision`, parser —execution boundaries have become a small policy language rather than a handful of `if / else` checks. And it is not rhetorical: schemas mark required fields and disable stray properties via `additional_properties = false`; `execpolicy/src/lib.rs` exports parser, `PolicyParser`, plus helpers like `blocking_append_allow_prefix_rule` and `blocking_append_network_rule`. Codex evaluates policy and also amends it in structured ways.

4.4 Runtime approvals versus policy language

Claude Code leans toward a runtime approval chain; Codex leans toward explicit policy language and parameterized approvals. Claude Code's ask/allow/deny is tightly coupled to the call site, deciding by context, tool type, user action, and session state —sensitive, but rules stay buried in runtime logic. Codex's exec-policy pulls rules outward as separately parsable and evaluable entities —better readable and portable, a natural fit for team governance; the cost is weight and real ongoing design work. Bluntly: shift manager making the call on site vs. a company that writes policy first and checks compliance second.

Skeleton: two risk gates

```
// skeleton: Claude Code runtime approval (src: src/hooks/useCanUseTool.tsx,
decision = hasPermissionsToUseTool(tool, input, ctx) // allow | deny | ask
match decision:
```

```
   allow: schedule_with_concurrency_check(tool)     // isConcurrencySafe()
    deny:  reject(reason)                                // sticky
    ask:   route_to(coordinator | swarm | interactive)
interrupt_semantics = tool.interruptBehavior ∈ {cancel, block}

// skeleton: Codex exec-policy evaluation  (src: execpolicy/src/lib.rs, local_
policy = PolicyParser.parse(source)
for rule in policy.rules:
    eval = rule.evaluate(exec_command { cmd, workdir, shell, tty,
                            yield_time_ms, max_output_tokens, login })
  if eval.matches: return Decision::{Allow | Deny | RequestPermissions}
return Decision::default
```

Approval decision tree

```
tool_call
  ├─ schema validation fails   → deny (additional_properties=false blocks stray
   ├─ matches deny rule (prefix) → deny
  ├─ matches ask rule (net/esc) → request_permissions (explicit tool)
  ├─ no match + sandbox relaxed → allow (bounded by workdir / sandbox mode)
   └─ no match + sandbox strict  → ask
```

Timeout and parameter thresholds

Name	Purpose	Source
`yield_time_ms`	max ms a single exec may block	`local_tool.rs (exec_command)`
`max_output_tokens`	cap on tool output admitted into context	`local_tool.rs`
`additional_properties=false`	blocks model from injecting stray args	`local_tool.rs (schema)`
Bash subcommand cap	max compound subcommands per Bash call	`bashPermissions.ts`

4.5 Sandbox and approval define the product

Sandbox, approval, and permission are not security accessories —for a coding agent they define what the product is. A system that lets the model run arbitrary commands in a user directory is an agent that transfers risk to the user, not just a "more powerful" one. Explicitly expressing sandbox mode, network access, approval policy, additional directories, state-store location, and MCP tool approvals is what delivers capability plus behavioral boundaries. Codex exposes these turn-level conditions in `thread.ts` as part of thread semantics; Claude Code pushes the boundary into runtime —tool handling, interrupts, permission hooks, Bash restrictions. One "executes while being watched," the other "declares the execution contract before starting."

4.6 MCP, external tools, and boundary migration

Both systems can attach more capabilities, but the difference remains. Claude Code weaves skills, hooks, permissions, and tool prompts into a situational governance chain so local rules ride the main loop. Codex pulls external capabilities into one unified tool system —MCP resources, dynamic tools, and tool discovery in `tools/src/lib.rs` expect extensions to become schema-defined, rule-governed tool objects rather than runtime understandings. Once the ecosystem grows, "how extensions obey the general rules" becomes the ballast: the team that thinks through boundary migration early keeps its extension ecosystem from degenerating into a junk closet.

4.7 Chapter conclusion

This chapter can be reduced to a hard sentence:

> Claude Code's tool governance depends more on runtime orchestration and situational approvals, while Codex's tool governance depends more on schemas, parameterized permissions, and an independent policy system.

The former is like an experienced foreman.

The latter is like a contractor with a compliance office and legal department.

If you look only at the fact that both can run commands, you miss the meaningful difference. The real question is who owns order before the tool starts moving.

The next chapter looks at a more grounded layer: skills, hooks, local rule files, and team institutions. Once a technical system has to enter a team, it eventually has to learn village law.

Chapter 5 Skills, Hooks, and Local Rules: How a System Learns to Respect Village Law

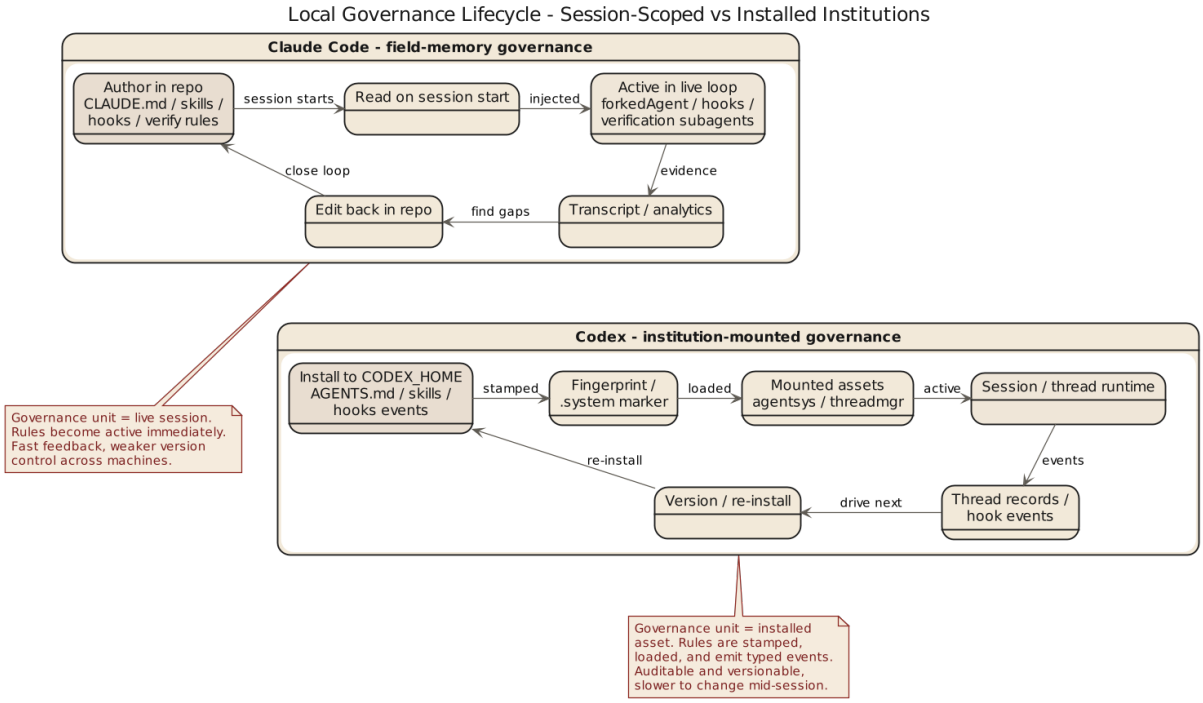

Figure 5: Local Governance Comparison

5.1 Any deployable agent becomes local

Any general-purpose coding agent that starts real work for a team collides with the same fact: companies, repositories, directories, and people all have their own rules and habits. A system that cannot absorb those local institutions stays trapped in demo environments. Claude Code and Codex both answer, in different directions.

5.2 Claude Code: local institutions as field memory

Localizing capacity lives in `CLAUDE.md`, skills, hooks, and session memory — together they feel like accumulated field experience: `CLAUDE.md` states what counts as common sense in this repo, directory, and team; skills package repeatable workflows; hooks attach team governance to lifecycle points; session memory keeps each turn from starting life over from zero. The common trait is proximity to the task scene —get rules into the current session and current execution, not a single eternal constitution first. Claude Code resembles an engineer who copies down local custom wherever it goes —highly practical across projects, directories, and local constraints, but without cleanup, knowledge expands as field patches.

5.3 Codex: local institutions as structured injection and event systems

Codex also has skills, local rules, and hooks, but the temperament is more institutional. Skills: `skills/src/lib.rs` installs system skills into `CODEX_HOME/skills/.system` and tracks them by fingerprint —a skill is an installed, managed, versionable asset, not text casually reread at startup. `install_system_skills()` reinstalls only when the marker fails to match. `AGENTS.md` carries scope and hierarchy rather than meaning "read a local note" —local rules carry positional relationships, not just content. Hooks: `hooks/src/engine/mod.rs` splits events into `session_start`, `pre_tool_use`, `post_tool_use`, `user_prompt_submit`, `stop`, and each handler has `event_name`, matcher, timeout, status message, source path, display order —more a formal lifecycle event system than a "drop a callback wherever convenient" pattern. The engine separates `preview_*` from `run_*` paths (preview which handlers fire before executing), and on Windows disables `codex_hooks` with an explicit warning when support is incomplete —hook capability is made explainable.

5.4 Claude Code absorbs experience; Codex mounts institutions

Claude Code's local governance continuously absorbs field experience into the neighborhood of the main loop —strong at making the agent learn quickly how things are done here. Codex mounts local rules onto an explicit control plane and lifecycle system —strong at making rules categorized, ordered, installed, and triggered. Team feel diverges: the former is an old employee who can read the room; the latter is a newcomer with strong institutional instincts who posts the rules first, then coordinates the work.

Skeleton: Codex hook lifecycle

```
// skeleton: Codex hook engine  (src: hooks/src/engine/mod.rs)
events = [session_start, user_prompt_submit, pre_tool_use, post_tool_use, sto
for ev in events:
   handlers = preview_handlers(ev, ctx)      // preview matches first
    emit(preview_event { ev, handlers })
    for h in handlers sorted by display_order:
        if match(h.matcher, ctx) and not timed_out(h.timeout):
            run_handler(h)                        // actually fire
        else:
            log_skip(h, reason)
on platform == windows: disable(codex_hooks); warn("incomplete support")
```

Invariants: hook event ordering

```
assert session_start fires once per thread before any tool_use
assert pre_tool_use fires immediately before execution; post_tool_use after
assert stop fires exactly once per thread termination path
assert preview_* path never executes handlers; only run_* does
assert each handler has {event_name, matcher, timeout, source_path, display_or
assert stable display_order ⇒ replayable ordering across runs
assert skill fingerprint mismatch ⇒ reinstall; match ⇒ skip (skills/src/lib
```

5.5 Different consequences for organizational reproducibility

A system that relies on injected field experience adapts faster to new repositories and stays effective in dense local context. But when it spreads across teams, it usually needs editorial cleanup —otherwise everyone writes their own `CLAUDE.md` and invents their own skills, and the organization ends up printing textbooks by province. A system that relies on structured injection and lifecycle mounting has more expansion potential: rules are easier to distribute uniformly, version, and audit. The cost is learning: the team must accept more explicit institutions first. Classic tradeoff —closer to the scene gives elasticity, more institutionalized gives reproducibility. What decides the outcome is which stability the team actually needs.

5.6 Chapter conclusion

This chapter can be summarized as:

> Claude Code tends to turn local governance into field memory and runtime injection, while Codex tends to turn local governance into structured assets and lifecycle event systems.

That is not just another way of saying "both support skills and hooks."

The difference is that Claude Code asks, "How do we make the agent work here more like a local?" while Codex asks, "How do we make local rules enter a governable institutional framework?"

The next chapter moves into a higher-risk layer: multi-agent work, verification, persistent state, and recovery. Once several agents begin working at once, rules alone are not enough; responsibility has to be partitioned too.

Chapter 6 Delegation, Verification, and Persistent State: Who Stops the System from Grading Itself

6.1 The real problem in multi-agent systems is responsibility

Hearing "multi-agent" and jumping to efficiency is like hearing a headcount plan —the truly difficult part is never more agents but how responsibility gets divided. If one system executes, summarizes, verifies, and casually writes its own review, the result is comforting and unreliable: "good job." Claude Code separates explore, execute, synthesis, and verification and treats verify as an independent discipline rather than a polite closing flourish —"done" is not declared by the executing agent alone. Codex walks the same road: `tools/src/lib.rs` exposes `create_spawn_agent_tool_v*`, `create_wait_agent_tool_v*`, `create_send_message_tool`, `create_close_agent_tool_v*` —delegation is formal tool capability, not runtime black magic.

6.2 Claude Code: multi-agent serves runtime separation of responsibility

The mechanism stays centered on the main loop and task progression: the primary agent should not do everything itself, and must not both implement and certify. Multi-agent handles outsourced exploration, split implementation, synthesis, and independent verification. That fits the strength in runtime orchestration —multi-agent enters the governance framework of "how the cur-

rent task moves forward," rather than a grand agent platform into which tasks are slotted.

6.3 Codex: multi-agent serves explicit tool-mediated collaboration

Delegation is defined as tool interface, pushing multi-agent toward a formal subsystem. Two effects: delegation becomes easier to record, audit, and compose because it is an explicit tool call, not hidden runtime maneuvering; collaboration aligns with threads, state, and approval systems, inheriting the first-class infrastructure Codex already built. In `agent_tool.rs`, `spawn_agent`, `send_input`, `wait_agent`, and `close_agent` each have their own schemas: `send_input` distinguishes `interrupt=true` from default queued delivery; `wait_agent` is parameterized with default/min/max timeouts; `close_agent` explicitly closes open descendants. Codex turns preemption, waiting, and cleanup into protocol fields —a strong foundation for platform capability, not always nimble but durable.

6.4 Persistent state keeps verification from becoming etiquette

Verification turns ceremonial mainly because state handoff is weak. What was done, why, which tools, which files —if that only lives in the executor's head, verification becomes serious-looking theater without material. Claude Code keeps session state, tool results, and recovery branches continuously visible inside runtime, then pairs that continuity with an independent verification discipline. Codex provides explicit material via thread, rollout, message history, and state DB bridge —a system with session-archive awareness is simply better at answering "what exactly happened just now." The two are not in conflict: Claude Code repairs executors too immersed in the scene; Codex repairs collaboration that must leave structured evidence.

6.5 Different attitudes toward recovery and closure

Claude Code cares deeply about task cleanup, parent-child abort propagation, and subagent lifecycle hooks —in its world multi-agent work is first a live runtime phenomenon that must close down quickly when the scene goes wrong. Codex leans the other way, bringing agent lifecycle under explicit state management and invocation protocol: it cares not only whether a subagent died, but how that delegated act should persist as a system event. One resembles a site chief worrying about holes left behind; the other resembles an organizer with project infrastructure worrying whether every collaboration act entered the record.

Skeleton: Codex delegation protocol

```
// skeleton: agent_tool.rs spawn/wait/send/close
handle = spawn_agent { role, prompt, timeout, inherit_approval }
for msg in updates:
   send_input(handle, msg, interrupt ∈ {true, false})  // true=preempt, false=q
result = wait_agent(handle, timeout ∈ [min, default, max])
close_agent(handle, cascade=true)                // closes open descendants t
```

Orphan and timeout failure matrix

Event order	Pre-state	Trigger	Next	Threshold
parent abort	child in flight	parent.abort	cascade abort to handle; write rollout event	—
`wait_agent` timeout	child unreturned	timeout ⩾ max	close handle, return timeout result	`wait_agent.max`
`send_input(interrupt=true)`	child queued	preemption	drop queue, inject new input	—

Event order	Pre-state	Trigger	Next	Threshold
`close_agent`	open descendants	explicit close	cascade-close all descendants	`cascade=true`
child crash	—	abnormal exit	return error, keep thread record	—
handle leak	task ends without close	finalize	force close + evict	no dangling handles

6.6 Chapter conclusion

The conclusion is not difficult to state:

> Claude Code' s multi-agent design emphasizes runtime separation of responsibility and field closure, while Codex' s multi-agent design emphasizes tool-mediated delegation, state handoff, and auditable collaboration.

Both are trying to stop the system from giving itself inflated grades.

Claude Code leans more on role separation and verification discipline.

Codex leans more on explicit interfaces, thread state, and collaboration records.

The final chapter compresses the previous six into one overall judgment and answers the book's title question directly: convergence through different roads, or genuinely different species?

Chapter 7 Convergence and Divergence

7.1 Start with the part where they converge

If I had to give the shortest conclusion first, I would say: yes, they genuinely converge.

The reason is straightforward. Neither Claude Code nor Codex treats the model as an executor worthy of direct trust. Both systems accept:

- prompt does not control everything
- tools must be constrained
- long sessions require state governance
- local rules must enter the system
- multi-agent execution requires role partitioning and verification

In other words, both have already moved beyond the naive stage where people imagine that stronger models will somehow dissolve systems problems by themselves. Once a system reaches this point, it no longer treats an agent as merely a chatbot with a few tools attached.

So at the level of direction, they do meet at the same destination: harness is the true control layer, and the model is the most productive but also most unstable component underneath it.

7.2 Now the part where they branch

But it would be far too rough to call them fundamentally identical.

Claude Code’s main axis looks something like this:

- begin from the query loop
- handle continuity in runtime
- preserve order with compaction, tool orchestration, interrupts, and recovery
- connect field rules and team institutions through skills, hooks, and verification

Codex’ s main axis looks more like this:

- begin from explicit module boundaries and explicit control layers
- turn instructions into fragments
- turn tools into schemas
- turn execution boundaries into policy
- turn sessions into thread / rollout / state
- turn local rules and hooks into structured assets and event systems

The first feels like a system grown from mechanical experience.

The second feels like a system grown from institutional design.

That is where the branching really lies. The difference is not destination. It is skeleton.

7.3 If I had to give them a harsher but more accurate label

I would even be willing to call them two different political forms of harness.

Claude Code is closer to a runtime republic. A great deal of power is concentrated in the main loop and field dispatch, and order is maintained through continuous negotiation with reality. It is not anti-institution; institutions just tend to exist in service of the live session.

Codex is closer to a constitutional control plane. Power is first written into types, fragments, policy, threads, and event systems. Runtime still judges, of course, but it judges inside a more explicit framework.

That is an exaggerated description, but it clarifies an important point: harness is never only a pile of technical parts. It is also a way of distributing power. Who

defines the boundary, who interprets state, and who owns the final authority of execution all eventually appear in architecture.

7.4 What later builders should learn

If a team intends to build its own coding agent, the true value of this comparison is not side-picking. It is avoiding two categories of mistake.

The first mistake is thinking a feature table is enough. It is not. A team has to decide what its primary contradiction is. Is the real problem that long sessions go out of control, or that rule sources are too scattered and permission boundaries remain unclear? Different contradictions push you toward different harness shapes.

The second mistake is trying to splice together the most attractive features of both systems without making actual tradeoffs. In engineering, what is most dangerous is often not tradeoff itself but the refusal to make one. If you want fully dynamic runtime flexibility and a completely explicit structured control plane at the same time, you usually end up with neither.

The saner approach is:

- if you fear loss of control on site, strengthen the runtime heartbeat first
- if you fear institutional drift, make instruction, tool, policy, and state explicit first
- once the primary contradiction stabilizes, gradually build the opposite side

One more caution belongs here. In practice many younger systems do neither job fully. They neither harden runtime discipline nor make the control layer truly explicit. Instead they take a third, easier-looking road: keep stuffing more bootstrap files, role descriptions, skill explanations, and workspace text into prompt, hoping that informational fullness will compensate for a weak skeleton. Such systems often appear workable in the short term, but over longer sessions they expose the same double failure: tokens are expensive, and working semantics are still unstable.

7.5 Final judgment

The question in the title can now be answered directly.

They are convergence through different roads.

They are also distinct branches of the same larger problem.

“Convergence” names the reality they both accept: the model is unreliable, and harness is where order comes from.

“Different branches” names the different political economy through which that reality is implemented: Claude Code trusts runtime discipline more, while Codex trusts explicit control layers more.

Systems that rely mainly on prompt stacking to hold context together sit in a less settled middle zone. They have already realized that models forget and drift, so they add memory, skills, and compaction. But if context governance still follows a “inject first, rescue later” logic, then the system still has not decided clearly which layer should own order.

Neither path is inherently nobler. The real question is this: in which layer is your system prepared to cage uncertainty?

The location of that cage determines what the system will later become.

Chapter 8 If You Need to Build Your Own Harness: Whom to Learn From, and What to Learn First

8.1 The real use of comparison is to avoid unnecessary detours

The least interesting ending is the consumerist one —choose A or B. Engineering systems are not headphones and do not ship through ranking charts. The useful question is: if you are about to build your own harness or refactor an existing agent, whose lessons should you learn first, and which part first. Claude Code and Codex offer two opening moves, not two final answers. Claude Code warns you not to make runtime problems sound too elegant —what drags systems down are the dirty jobs inside the query loop (closing tool-result ledgers, managing context inflation, recovering from interrupts, cleaning up subagents, keeping verification independent, preventing failure handling from looping on itself); treating those problems lightly gives a system that looks smart and feels lethal to operate. Codex warns you not to let the control layer dissolve into tacit understanding —instruction sources, tool schemas, approval policy, thread state, hook events, and skill assets become easier to govern the earlier they are made explicit; teams that keep asking runtime improvisation to replace institution eventually find their system resembles a shed built out of verbal agreement.

8.2 Three common team shapes, three opening directions

Type one: the team already has an agent prototype, but long sessions frequently lose control

This kind of team usually needs Claude Code first.

Their problem is rarely that "the control plane is underdefined." Their problem is that the system cannot stay alive long enough. Typical symptoms include:

- context gets noisier over time
- tool-call chains break
- state becomes unclear after interruption
- no one closes subagent work cleanly
- verification degenerates into something people merely say

When those are your symptoms, the first thing to repair is runtime heartbeat, not more configuration. Stabilize the loop first. Institutional aesthetics can wait.

Type two: the team already has many rules, but rule sources are scattered and permission boundaries are unclear

This kind of team usually needs Codex first.

Their problem is not that the live scene is impossible to survive. Their problem is that the system becomes harder and harder to govern. Common symptoms include:

- local rules are scattered everywhere
- no one can say which constraints live in prompt and which live in tools
- approval logic is mixed into code and hard to explain
- once multiple extensions are introduced, boundaries blur further

What these teams need is an explicit control layer. Turn instruction, tool, policy, and thread into explicit concepts before asking runtime to work inside them.

Type three: there is no mature system yet and the team is starting from scratch

This is the most dangerous case, because it is easiest to envy the strengths of both sides at once and build a failed compromise.

A steadier route is usually:

- choose one primary contradiction
- design the main skeleton around that contradiction
- add the opposite side only at minimum level for now

If your first-phase risk is mainly "the model will behave recklessly," start with runtime discipline in the Claude Code sense.

If your first-phase risk is mainly "the team will lose institutional order," start with an explicit control layer in the Codex sense.

The worst move is trying to learn both sides fully at the same time and ending up with neither a stable main loop nor a clear control plane.

Staged builder checklists for the three team shapes

```
# Type one: prototype exists, long sessions lose control (learn Claude Code fir
- [ ] Week 1: name the loop state set {messages, toolUseContext, compactTrackin
- [ ] Week 1: every tool_use closed or synthetic-filled; abort path wired
- [ ] Week 2: context governance trio — memory / collapse / autocompact thresho
- [ ] Week 2: verification independent of implementation (verifier ≠ implemente
- [ ] Week 3: subagent lifecycle SubagentStart/Stop observable
Gate: 24h continuous session without token breaker, orphan subagents, or tool_

# Type two: rules multiply, sources scattered, boundaries unclear (learn Codex
- [ ] Week 1: all instructions become fragments — marker, source, precedence al
- [ ] Week 1: tools schema-typed with additional_properties=false
- [ ] Week 2: approval policy lifted into rules — deny/ask/allow independently
- [ ] Week 2: thread.id / rollout established; turn-level {approvalPolicy, sand
- [ ] Week 3: hooks split into pre/post/session_start/stop; skill assets instal
Gate: any rule change lands via PR diff alone, no runtime code edits required
```

```
# Type three: starting from scratch (pick one primary contradiction first)
- [ ] Week 1: declare primary contradiction — "model runs wild" or "team loses
- [ ] Week 1: define minimum permission model (deny/ask list for high-
risk actions)
- [ ] Week 2: stand up skeleton on the primary side (loop OR fragment+thread, pi
- [ ] Week 3: bring the other side up to minimum-viable only (recovery path OR ba
- [ ] Week 4: land 1–2 skills/tools; prove the loop closes end to end
Gate: a new member can advance the checklist without verbal tutoring from the o
```

8.3 What to learn from Claude Code, and what to learn from Codex

Prioritize Claude Code on: the state mind-set of the query loop, compaction and context governance, tool orchestration and interrupt handling, subagent lifecycle and verification independence, treating failure paths as main paths. Prioritize Codex on: instruction fragmentation, tool schemas, explicit expression of approval and policy, infrastructure for thread / rollout / state, hook events and skill-asset management. This is not compromise —it assumes you know what you are learning. The right reason to borrow something is that it repairs your weakness, not that someone else already built it.

Carry the context-governance judgments from the first volume into third-party harnesses and a common but costly route becomes easy to identify. It does not separate context into units with different lifetimes, duties, and entry costs. Instead it packs bootstrap files, skill descriptions, identity settings, and workspace text into prompt as far as possible, then leans on truncation, compaction, and recovery chains once the window gets tight. These systems may still have memory, skills, compaction, even upper bounds —the governing axis remains "inject first, rescue later." Once context is organized mainly by piling up text, token waste is only the first cost; signal dilution is the deeper one: the model sees more, but is not necessarily clearer about which working semantics matter next.

Three routes in one line: Claude Code treats context as working memory (what must survive, what should be compressed); Codex treats context as structured units (source type, scope, state handoff); the OpenClaw family treats context as a prompt container (what else can still be packed in before the limit). That is

why teams on the third route feel "more informed" at first, then complain about two things at once —tokens burn fast and quality does not climb as context fattens. It is solving how much can be inserted, not what must be preserved for continued work.

8.4 One dangerous misconception: explicitness and flexibility are not natural enemies

System builders often rely on a lazy false opposition. Say "explicit control layer" and they imagine a heavy, slow, rigid system; say "runtime flexibility" and they imagine experience can carry the system now while structure is deferred. Neither instinct is intelligent. Explicitness is not inherently rigid, and flexibility is not inherently chaotic. The real question is whether you have defined clearly which things must be explicit and which can be left to field judgment. Claude Code's strength is not rejecting structure but knowing which troubles must be faced inside runtime; Codex's strength is not rejecting flexibility but knowing which boundaries will turn into endless disputes if not declared early. A good third harness does not average the two —it distinguishes which rules must be written down first, which judgments can remain in runtime, which state must persist, and which experience only needs to live inside session memory.

8.5 A practical order of operations for later builders

From zero, the recommended sequence: (1) high-risk actions and the minimum permission model; (2) main loop or thread lifecycle; (3) context governance and recovery paths; (4) skills, local rules, and hooks; (5) multi-agent, platform capability, and complex ecosystem. Not glamorous, but roughly the order in which incidents appear. In engineering, many design orders should follow the order of failure, not the order of demo aesthetics.

8.6 Chapter conclusion

This chapter only wants to leave one plain sentence behind:

> Learn from Claude Code mainly to understand how a system stays stable on site; learn from Codex mainly to understand how a system sustains order over time inside an organization.

Teams that learn only the first tend to become rich in experience and poor in institution.

Teams that learn only the second tend to produce elegant institutions and fragile field behavior.

The better move is not side-picking. It is deciding which bone your system must grow first according to its primary contradiction.

Appendix A Source Map: Which Files This Comparison Primarily Relies On

This appendix does one thing only: it states which files each chapter's judgments mainly rely on. It is not a directory of reproduced source, nor does it imply that the book ships source code together with its arguments.

The same boundary applies here:

- only the minimum engineering citation and module location is used
- the body of Claude Code or Codex implementation is not reproduced
- no long source transcription is provided

A.1 Primary references on the Claude Code side

Overall control plane and prompt:

- `src/constants/prompts.ts`
- `src/utils/systemPrompt.ts`
- `src/utils/claudemd.ts`
- `src/memdir/memdir.ts`

Runtime loop and recovery:

- `src/query.ts`
- `src/QueryEngine.ts`
- `src/services/compact/autoCompact.ts`

- `src/services/compact/compact.ts`

Tools and permissions:

- `src/services/tools/toolOrchestration.ts`
- `src/services/tools/toolExecution.ts`
- `src/services/tools/StreamingToolExecutor.ts`
- `src/hooks/useCanUseTool.tsx`
- `src/tools/BashTool/prompt.ts`
- `src/tools/BashTool/bashPermissions.ts`

Multi-agent work, skills, and hooks:

- `src/utils/forkedAgent.ts`
- `src/coordinator/coordinatorMode.ts`
- `src/tasks/LocalAgentTask/LocalAgentTask.tsx`
- `src/tools/SkillTool/SkillTool.ts`
- `src/tools/SkillTool/prompt.ts`
- `src/utils/hooks/hooksConfigManager.ts`

A.2 Primary references on the Codex side

Core module skeleton:

- `core/src/lib.rs`
- `tools/src/lib.rs`
- `skills/src/lib.rs`
- `hooks/src/lib.rs`

Instruction fragments and user injection:

- `instructions/src/lib.rs`
- `instructions/src/fragment.rs`
- `instructions/src/user_instructions.rs`
- `docs/agents_md.md`

Tools, approvals, and execution policy:

- `tools/src/local_tool.rs`
- `tools/src/agent_tool.rs`
- `execpolicy/src/lib.rs`
- `docs/execpolicy.md`
- `docs/sandbox.md`

Threads and state:

- `sdk/typescript/src/thread.ts`
- `core/src/lib.rs`
- `core/src/thread_manager.rs`
- `core/src/rollout.rs`
- `core/src/state_db_bridge.rs`
- `core/src/message_history.rs`

Hook event engine:

- `hooks/src/engine/mod.rs`

A.3 Chapter-to-file mapping

Chapter 1:

- Claude Code's `query.ts`, `toolOrchestration.ts`
- Codex's `core/src/lib.rs`

Chapter 2:

- Claude Code's prompt assembly and `CLAUDE.md`
- Codex's `fragment.rs`, `user_instructions.rs`

Chapter 3:

- Claude Code's query loop and `QueryEngine`
- Codex's `thread.ts`, plus exposed modules around `thread_manager`, `rollout`, and `state_db_bridge`

Chapter 4:

- Claude Code’s tool orchestration, Bash restrictions, and permission semantics
- Codex’s `tools/src/lib.rs`, `local_tool.rs`, and `execpolicy/src/lib.rs`

Chapter 5:

- Claude Code’s skill, hook, and memory system
- Codex’s skills system and hook engine

Chapter 6:

- Claude Code’s forked-agent design and verification discipline
- Codex’s agent-tool set and its thread / rollout / state structures

Chapter 7:

- the cumulative judgment produced by the whole file set above

Appendix B Checklists: How to Tell Whether Your Harness Resembles Claude Code, Codex, or an Unfinished Prototype

If comparison cannot be reduced into checklists, what remains is usually a set of well-phrased but hard-to-use conclusions. This appendix compresses the previous chapters into questions a team can actually discuss.

B.1 Control-plane checklist (invariants)

```
assert every instruction has {source, type, precedence}      # fragments are ic
assert prompt separates control plane from output style      # tone ≠ order
assert local-rule scope (CLAUDE.md / AGENTS.md) explicitly labeled
assert team-rule changes land via diff                  # not oral agreement
```

If these cannot be answered, the control plane is still at "good enough to use" rather than "good enough to govern."

B.2 Continuity checklist (invariants)

```
assert continuity sovereignty ∈ {main loop, thread+rollout+state}  # pick one e
assert interrupt ⇒ tool_result closed (synthetic fallback counts)
assert long session has compact / truncation / recovery trio
assert thread.id / session indexing / persisted state = first-
class concepts
```

If long sessions rely on the model to "remember," you do not need the rest of the evaluation.

B.3 Tool and approval checklist (invariants)

```
assert tool = schema-typed interface, additional_properties=false
assert approval policy independently evaluable (not buried in code if/else)
assert high-risk tools (Bash etc) get dedicated governance     # not flat treat
assert {workdir, network, sandbox, approval} explicitly expressible
```

If the only answer is "we also have permission controls," the permission system has not been designed.

B.4 Local governance checklist (invariants)

```
assert local rules layerable by {directory, team, task type}
assert skill = reusable institutional slice, not long prompt   # has version / s
assert hooks attach to explicit lifecycle events (pre/post/session_start/stop
assert {skill, rule, hook} carry {version, source, trigger boundary}
```

B.5 Multi-agent and verification checklist (invariants)

```
assert multi-agent's first purpose is responsibility split; parallelism is a bo
assert independent verifier exists (verifier ≠ implementer)
assert delegation = explicit tool or explicit state event, not runtime magic
assert child-agent {failure, timeout, cancel} ⇒ named cleanup owner
```

B.6 Which kind of system are you closer to?

Signals that you are closer to Claude Code:

- you care most about query loop, tool orchestration, interrupts, compaction, and recovery
- you are good at getting rules into the live session quickly
- you care primarily about how an agent keeps running inside complex tasks

Signals that you are closer to Codex:

- you care most about instruction fragments, tool schemas, approval policy, thread / rollout / state
- you are good at turning local rules into structured assets
- you care primarily about how an agent is governed durably inside an organization

Signals that you are closer to an unfinished prototype:

- you can recite vocabulary from both sides, but cannot explain who owns order
- you have many capability entry points, but no clear recovery path
- you have many rule texts, but no scope or precedence
- you have multi-agent execution, but no separation of responsibility and no closure mechanism

B.7 Six final questions

If time is short, ask only these six:

- Who owns the final control, the model or the harness?
- Does continuity live mainly in the loop, or in threads and state?
- Before tools act, who stops the last dangerous move?
- How do local rules enter the system, and how are they layered?
- Who owns verification, and how is it kept independent?
- After something goes wrong, what evidence lets the team trace the path back?

Once these six questions are asked, the system's political family usually reveals itself.

B.8 Thresholds & orderings quick reference

Name	Value	Purpose	Source
`MAX_ENTRYPOINT_LINES`	200	entry file line cap	Book 1 ch5 / `memdir/memdir.ts`
`MAX_SECTION_LENGTH`	2_000	session-memory per-section cap	`SessionMemory/prompts.ts`
`MAX_TOTAL_SESSION_MEMORY_TOKENS`	12_000	session-memory total budget	`SessionMemory/prompts.ts`
`AUTOCOMPACT_BUFFER_TOKENS`	13_000	autocompact warning buffer	`compact/autoCompact.ts`
`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES`	3	breaker threshold	`compact/autoCompact.ts`
`yield_time_ms`	per-tool	max ms a single exec may block	`local_tool.rs`
`wait_agent.timeout`	min/default/max	child-agent wait window	`agent_tool.rs`
Bash subcommand cap	implicit	max compound subcommands per call	`bashPermissions.ts`

Event orderings:

- session_start → user_prompt_submit → pre_tool_use → tool exec → post_tool_use → stop
- spawn_agent → send_input* → wait_agent → close_agent (cascades to descendants)
- PTL → collapse → reactive compact → if still PTL, surface error (no further loop)